

Shaurya Doger

Experiment 4

04/07/2023

Professor Jae rock Kwon

Experiment 4 Report

This report below will explain my results regarding the “**Experiment-4**” results including SysTick Timer, Interrupt Handler, Interrupt Enable, and Background Software Tasks:

- 1) The screenshots below will show my SysTick Timer Setup, Interrupt Handler, and Enabled Interrupts:

```
64 // SysTick timer initialization
65 SysTickEnable();
66
67 //80MHz clock generates 800000 ticks per 10ms
68
69 SysTickPeriodSet(SysCtlClockGet()/1000); // 1 ms
70 SysTickIntRegister(&SysTick_Handler);
71 SysTickIntEnable(); // interrupt enable
72
```

- a. The code shown above is used to initialize “**SysTick Timer**” to call the interrupt handler every “**1 ms**” along with registering “**SysTick_Handler()**” as the handler. We also want to enable the interrupt as all the above code is within the “**main**” function.

```
128 void SysTick_Handler(void)
129 {
130     timer_count++;
131     ui32TempValueC = getCelsiusValue();
132
133     unsigned char red_led_status;
134
135     //GPIO_PORTF_DATA_R ^=0x02;
136
137     red_led_status = GPIOPinRead(GPIO_PORTF_BASE, RED);
138
139     if(red_led_status == 0)
140         GPIOPinWrite(GPIO_PORTF_BASE, RED, RED);
141     else
142         GPIOPinWrite(GPIO_PORTF_BASE, RED, 0x00);
143
144 }
145
```

- b. The above screenshot illustrates functionality behind “**SysTick_Handler()**” in that every time it gets called, “**timer_count**” variable increments as the “**Celsius temperature**” also gets read into global variable “**ui32TempValueC**”. Aside from that, “**RED_LED**” also gets turned on as well for status indication.

```

187//This function prints the two passed temperatures to the console in an
188//easy human readable format.
189void PrintTemps (uint32_t TempC, uint32_t TempF)
190{
191    //SysTickIntDisable();
192    seconds = timer_count/1000;
193    remaining_milliseconds = timer_count % 1000;
194
195    minutes = seconds/60;
196    seconds = seconds % 60;
197
198    hours = minutes/60;
199    minutes = minutes % 60;
200
201    //UARTprintf("%2d:%2d:%2d:%2d\t%3d*C\t%3d*F\n", hours, minutes, seconds, remaining_milliseconds, TempC, TempF);
202    UARTprintf("%2d:%2d:%2d:%2d\t%3d\t%3d\n", hours, minutes, seconds, remaining_milliseconds, TempC, TempF);
203
204    //SysTickEnable();
205    //UARTprintf("%3d*C\t%3d*F\n", TempC, TempF);
206}

```

- c. The above code for printing temperatures is the main task along with the timings onto the “CoolTerm” as the shared variable includes “**timer_count**” and “**Celsius_Temperature**” in which both “**background task**” and “**ISR**” communicate with each other because those two functions utilize the same variable. “**Celsius_Temperature**” is utilized when we need to convert to “**Fahrenheit**”.
- 2) The correct implementation of setting up the interrupts involves the following:
 - a. Specify the interrupt type with either “**GPIO-Port**”, “**SysTick**”, or any other system as to how it should be triggered.
 - b. Create an ‘**Interrupt-Handler**’ function that utilizes shared-data variables to properly update essential information.
 - c. Register that “**handler**” into the built-in function through usage of.
 - d. Enable the “**interrupts**” to ensure that initialization is fully complete.
- 3) The below screenshot will show my printout of both “**temperatures**” along with timestamps in hours, minutes, seconds, and milliseconds:

```

ECE505 Lab 4: ADC and Interrupts
*****
Analog Input: Internal Temperature Sensor
Initialization Complete...
0: 0: 0:148. 17. 62
0: 0: 0:219. 18. 64
0: 0: 1:40. 18. 64
0: 0: 1:92. 18. 64
0: 0: 2:168. 18. 64
0: 0: 2:221. 18. 64
0: 0: 3:42. 17. 62
0: 0: 4:118. 18. 64
0: 0: 4:172. 18. 64
0: 0: 5:249. 18. 64
0: 0: 5:46. 18. 64
0: 0: 6:122. 18. 64
0: 0: 6:175. 18. 64
0: 0: 7:252. 18. 64
0: 0: 8:73. 18. 64
0: 0: 8:125. 18. 64
0: 0: 9:202. 18. 64
0: 0: 9:255. 18. 64
0: 0:10:76. 18. 64
0: 0:10:128. 18. 64
0: 0:11:205. 18. 64
0: 0:12:26. 18. 64
0: 0:12:78. 18. 64
0: 0:13:154. 18. 64
0: 0:13:207. 18. 64
0: 0:14:28. 18. 64
0: 0:14:80. 18. 64
0: 0:15:156. 18. 64
COM7 / 9600 8-N-1
Disconnected

```

- a. As per my observations, I am not currently aware of any “**shared data**” related issues that are occurring in terms of “**temperature**” and “**timing**” variables that printout the data to left. Timings are printing out in correct order as needed with “**counters**” and per “**ms**”.
- b. Even if such problems did occur because of updating “**timer_count**” during the time calculations for printing out “**UART**” values, we can disable the “**ISR**” before the start of calculation and re-enable them after operation is completed. Such solutions can be involved in “**PrintTemps()**” function.

- 4) Overall, this final experiment of the course was very challenging in many aspects including learning many concepts behind **“interrupts”**, **“Handlers”**, **“Timers”**, **“Data Sharing”**, etc. When executing this project, I did spend some time researching the techniques behind how such systems operate all together, especially when determining where functions (such as initialization) belong. Other challenges included setting up timer correctly to call **“Handler”** every **“millisecond”**, along with initializing variables in correct places to prevent **“data-sharing”** issues of incorrect values getting displayed. Even from all the difficulties of this experiment, I have still managed to get my code to work as well as print out correct **“temperature”** and **“timing”** values into UART (**CoolTerm**). I feel like that I can apply **“Interrupts”** across many other projects including **“robotics”**, **“remote-controlled cars”**, **“weapon-control-systems”**, and many others that I would be very interested in pursuing both academically and professionally.
- 5) YouTube video link will show my demonstration of this project:
<https://www.youtube.com/watch?v=xXk89L7NG74>